

Operadores en Python 🌟

Aprendimos que las `variables` nos permiten almacenar diferentes tipos de datos, funcionando como contenedores de información. Sin embargo, no basta con solo guardar estos valores, nuestra verdadera misión es interactuar con ellos y transformarlos en cada paso de nuestros programas.

Para lograr esta transformación, vamos a utilizar los `operadores`. En esta sección, aplicaremos operaciones matemáticas que ya conocemos y descubriremos algunas herramientas nuevas y sumamente potentes que `Python` nos ofrece para manipular estos datos.

Operaciones aritméticas

Suma +

El primero de estos operadores es la suma. En `Python` utilizamos el símbolo `+` para combinar valores. Nos sirve, por ejemplo, para acumular puntos o calcular totales.

```
1 income = 400
2 extras = 65
3
4 # Sumamos ambos valores para obtener el total
5 total_sum = income + extras
6
7 # Mostramos el resultado
8 print(total_sum)
```

[Copiar](#)

✓ 465

Resta -

Para encontrar la diferencia entre dos números, utilizamos el operador `-`.

```
1 initial_budget = 1000
2 total_expenses = 450
3
4 # Restamos el valor de la variable 2 a la variable 1
5 remaining_balance = initial_budget - total_expenses
6
7 # Mostramos el valor restante
8 print(remaining_balance)
```

[Copiar](#)

✓ 550

Multiplicación *

Para realizar la multiplicación de dos factores utilizamos el carácter asterisco `*`.

```
1 unit_price = 20
2 product_quantity = 5
3
4 # Calculamos el total multiplicando
5 total_cost = product_quantity * unit_price
6
7 # Mostramos el costo final
8 print(total_cost)
```

[Copiar](#)

✓ 100

División /

Cuando usamos el operador `/`, realizamos una división. Es importante recordar que en `Python`, este operador siempre nos devolverá un número decimal (un `float`), **incluso** si el resultado es exacto.

```
1 pizza_slices = 25
2 party_members = 4
3
4 # Calculamos los slices por persona
5 slices_per_member = pizza_slices / party_members
6
7 # Mostramos el resultado
8 print(slices_per_member)
```

Copiar

 6.25

En algunas ocasiones, necesitamos hacer una división ignorando los decimales, donde queremos el cociente **entero**. Para esto, utilizamos el operador de división entera `//`.

```
1 pizza_slices = 25
2 party_members = 4
3
4 # Calculamos los slices enteros por persona
5 slices_per_member = pizza_slices // party_members
6
7 # Mostramos el resultado
8 print(slices_per_member)
```

Copiar

 6

El ejercicio anterior obtenemos como resultado `6` ya que al hacer la división, el cociente entero de realizar la división de `25` entre `4` es `6`. Tomar en cuenta que esto **no es** un redondeo. Si el resultado es `1.99` terminará como `1`.

Módulo `%`

El operador `%` nos permite obtener el "sobrante" de una división entera. Es extremadamente útil, por ejemplo para determinar si un número es par o impar o para conversión de unidades.

```
1 total_items = 17
2 box_capacity = 3
3
4 # Obtenemos lo que sobra después de llenar las cajas
5 leftover_items = total_items % box_capacity
6
7 # Mostramos el residuo de la operación
8 print(leftover_items)
```

Copiar

✓ 2

El ejercicio anterior obtenemos como resultado `2` ya que al hacer la división, el residuo de realizar la división de `17` entre `3` es `2`.

Potencia ^

Para realizar cálculos exponenciales, como elevar un número al cuadrado o al cubo, utilizamos el doble asterisco `**`.

```
1 base_number = 2
2 exponent_value = 3
3
4 # Elevamos la base al exponente (2 al cubo) 2 * 2 * 2
5 calculation_result = base_number ** exponent_value
6
7 # Mostramos el resultado de la potencia
8 print(calculation_result)
```

Copiar

✓ 8

Resumen

Operación	Símbolo	Descripción	Ejemplos
Suma	<code>+</code>	Suma dos números	<code>10 + 35 → 45</code>
Resta	<code>-</code>	Resta un numero de otro	<code>90 - 50 → 40</code>
Multiplicación	<code>*</code>	Multiplica dos números	<code>3 * 5 → 15</code>
Division	<code>/</code>	Divide un numero por otro	<code>10 / 3 → 3.33</code>
Division entera	<code>//</code>	Divide un numero por otro pero quitando los decimales del resultado	<code>10 // 3 → 3</code>
Potencia	<code>**</code>	Eleva un numero a la potencia de otro	<code>2 ** 3 → 8 (2 * 2 * 2)</code>
Mod	<code>%</code>	Divide un numero por otro pero retorna su residuo (como el que vimos en Pseudocódigo)	<code>10 % 3 → 1</code>

Operaciones con strings

Antes de concluir con la lección, hay que mencionar que hay funcionalidades dentro de `Python` como "sumar" `strings`, lo cual es súper importante que conozcamos. A este proceso de unir dos o más `strings` lo llamamos **concatenación** y lo podemos conseguir de múltiples maneras.

Concatenación de Strings

La primera forma de lograrlo es mediante el operador de suma. Para ello, unimos múltiples `strings` y los unimos en un `string` resultante. A esto le llamamos concatenación.

```
1 first_word = "Hola"
2 second_word = "Mundo"
3
4 # Unimos las palabras y un espacio en blanco con el símbolo +
5 full_greeting = first_word + " " + second_word
6
7 # Mostramos el texto unido
8 print(full_greeting)
```

[Copiar](#)

✓ Hola Mundo

Format Strings `f-string`

La segunda forma es mediante `f-strings`, los cuales son la forma más moderna y elegante de unir variables dentro de un texto. Para crear un `f-string` solo necesitamos anteponer la letra `f` al inicio de las comillas, y agregar las variables dentro de llaves `{}`.

```
1 user_name = "Andrés"
2 user_age = 26
3
4 # Creamos el mensaje usando f-strings para unir los valores
5 welcome_message = f"Hola {user_name}, tu edad es {user_age} años"
6
7 # Mostramos el mensaje formateado
8 print(welcome_message)
```

Copiar

✓ Hola Andrés, tu edad es 26 años.

Reemplazo de llaves `string.format()` 🔑

Este es el método tradicional de unir `strings`. Utilizamos llaves `{}` como espacios reservados que luego serán rellenos por los valores que pasemos dentro de `.format()`.

```
1 item_name = "PC Gamer"
2 item_price = 800
3
4 # Formateamos el texto con el método format
5 price_tag = "El {} tiene un costo de {} USD.".format(item_name,
6
7 # Mostramos el mensaje final
8 print(price_tag)
```

Copiar

✓ El PC Gamer tiene un costo de 800 USD.